

CProject: Milestone 2 Deliverables and Checklist
Hardware for Artificial Intelligence and Machine Learning (HW4AI)
ECE 410/510
Spring 2026

Milestone 2 goals

Milestone 2 is where your project moves from analysis to working RTL. By the M2 deadline, the compute core and the interface module must each exist as synthesizable HDL with a testbench that produces a verifiable PASS against an independent reference, and your numerical precision choice must be documented with error analysis where applicable. M2 is the backstop against discovering non-synthesizable constructs, protocol violations, or unworkable precision choices during M3 synthesis.

How to submit

All M2 deliverables must be submitted by pushing to your public GitHub repository. There is no Canvas upload. The grader will scrape your repository at 11:59 pm on the due date and check for each file at the exact path listed in this checklist.

Expected repository structure for M2

Your repository should contain at minimum the following layout after M2. Paths from M1 must remain in place; M2 adds a new project/m2/ folder. The grader will check both M1 and M2 paths.

Note: Depending on the tools you are using and on your project specifics, your file extensions may be different. Please respect the repository structure as closely as you can.

```

your-repo/
├── README.md
├── project/
│   ├── heilmeyer.md
│   ├── m1/                                ← from M1, must still be present
│   └── m2/
│       ├── rtl/
│       │   ├── compute_core.sv            ← compute core HDL
│       │   └── interface.sv              ← interface module HDL
│       ├── tb/
│       │   ├── tb_compute_core.sv        ← compute core testbench
│       │   └── tb_interface.sv          ← interface testbench
│       ├── sim/
│       │   ├── compute_core_run.log      ← simulation transcript, compute
│       │   ├── interface_run.log         ← simulation transcript, interface
│       │   └── waveform.png              ← representative waveform
│       ├── precision.md                  ← num. format choice + error analysis
│       └── README.md                     ← how to reproduce M2 simulation
└── codefest/

```

The project/m2/ folder is new. Create it now if it does not exist. The exact filenames inside rtl/ and tb/ are required as written; the grader matches on filename. If your design uses additional source files, place them alongside and reference them from the testbench, but the two files named above must exist and must be the top-level modules.

Deliverable checklist

Check off each item only when the file is committed and pushed. A checked box that points to a missing or empty file is still Not Yet. The grader runs an automated check on every path; build artifacts and logs must be committed so the grader can read them without re-running your simulation.

Deliverable	GitHub file path
1. Compute core HDL	
☐ Compute core described in synthesizable SystemVerilog (or Verilog/VHDL) <i>Top module name must match the filename. No behavioral constructs that are not synthesizable (e.g., delays, file I/O, dynamic arrays in synthesizable scope).</i>	<code>project/m2/rtl/compute_core.sv</code>
☐ Module ports documented in a header comment block <i>Each port: name, direction, width, purpose. Reviewer reads this before the body.</i>	<code>project/m2/rtl/compute_core.sv</code>
☐ All clocked logic uses a single clock domain or documents any crossings explicitly <i>If you have more than one clock, name them and state the synchronization strategy.</i>	<code>project/m2/rtl/compute_core.sv</code>
☐ Reset behavior specified (synchronous or asynchronous, active level) <i>Stated in the header comment and consistent across the module.</i>	<code>project/m2/rtl/compute_core.sv</code>
2. Compute core testbench	
☐ Testbench drives at least one representative input vector <i>"Representative" means an input that exercises the dominant kernel from your M1 profiling, not a trivial all-zeros case.</i>	<code>project/m2/tb/tb_compute_core.sv</code>
☐ Expected output computed independently and compared to DUT output <i>Reference value must come from a software model (Python, C, or hand calculation), not from a prior run of the DUT.</i>	<code>project/m2/tb/tb_compute_core.sv</code>
☐ Testbench prints PASS or FAIL based on the comparison <i>A grader running your sim must be able to read PASS/FAIL from the log without inspecting waveforms.</i>	<code>project/m2/tb/tb_compute_core.sv</code>
☐ Simulation log committed showing PASS <i>Plain text. Must include the testbench's own PASS/FAIL line.</i>	<code>project/m2/sim/compute_core_run.log</code>
☐ Representative waveform image committed <i>PNG or PDF showing input application, internal pipeline activity, and output capture for the test vector. Annotate signals.</i>	<code>project/m2/sim/waveform.png</code>
3. Interface module HDL	
☐ Interface module described in HDL, matching the protocol selected in M1	<code>project/m2/rtl/interface.sv</code>

If you change the interface from your M1 selection, update project/m1/interface_selection.md and explain the change in project/m2/README.md.

- ☐ Module conforms to the published specification of the chosen protocol

SPI: clock polarity/phase stated. AXI4-Lite: handshake on every channel. AXI4 Stream: TVALID/TREADY contract honored. PCIe/UCle: cite the spec section the module implements.

- ☐ Register map or transaction format documented in a header comment

What address space, what each address does, what data widths.

project/m2/rtl/interface.sv

project/m2/rtl/interface.sv

4. Interface testbench

- ☐ Testbench exercises at least one complete write transaction

Full handshake including any required ready/valid signaling. Assert correctness of internal state after the write.

- ☐ Testbench exercises at least one complete read or response transaction

Verify the value returned matches what was written or what the protocol specifies should be returned.

- ☐ Testbench prints PASS or FAIL based on the comparison
- Same rule as the compute core testbench: graders read the log, not the waveform.*

- ☐ Simulation log committed showing PASS

Plain text with the PASS/FAIL line.

project/m2/tb/tb_interface.sv

project/m2/tb/tb_interface.sv

project/m2/tb/tb_interface.sv

project/m2/sim/interface_run.log

5. Precision and data format (OPTIONAL)

- ☐ Numerical format chosen and stated explicitly (FP32, FP16, BF16, INT8, INT4, fixed-point Q-format, custom)

If fixed-point, state integer bits, fractional bits, and rounding mode.

- ☐ Rationale for the choice grounded in your kernel and roofline

Why this format and not the next-narrower or next-wider one. Reference your M1 arithmetic intensity if precision affects byte traffic.

- ☐ Quantization error analysis if you use anything below FP32

Compare DUT output to FP32 reference on at least 100 samples. Report mean absolute error, max error, and (if classification) accuracy delta.

- ☐ Statement of acceptability with a threshold

"Error is acceptable because X" where X cites either a published baseline or an application-specific tolerance you justify.

project/m2/precision.md

project/m2/precision.md

project/m2/precision.md

project/m2/precision.md

- ☐ Document is ≥ 300 words

Below this length, the analysis is almost certainly missing one of the items above.

`project/m2/precision.md`

6. Reproducibility

- ☐ README documents how to run both testbenches
Exact command lines. State the simulator (Verilator, Icarus, ModelSim, Vivado XSIM, etc.) and version.
- ☐ Build instructions reproducible from a clean clone
If your testbench needs Python pre/post-processing, list the dependencies and pin versions.
- ☐ Any deviation from the M1 plan documented
Interface change, kernel scope change, precision change. State what changed and why.

`project/m2/README.md`

`project/m2/README.md`

`project/m2/README.md`